UNIVERSIDAD RAFAEL NÚÑEZ

Especialización en Analítica de Datos · Tecnologías de Procesamiento de Datos

Taller 1A en GitHub Codespaces — Hadoop y Spark: el mes de viajes

Docente: Heyder Medrano

Entorno: WEB: tu codespace de GitHub (4 núcleos, 16 GB), preparado con la «Preparación Previa — Taller 1 en GitHub Codespaces». Carpeta de trabajo: /workspaces/tpd-codespace/tpd.

Datos: tpd/datos/fhvhv_2025-06.csv: 19.868.009 viajes de Uber y Lyft en Nueva York, junio de 2025 (TLC), 25 columnas, 4,04 GB.

Objetivo técnico: Calcular por empresa los viajes, la tarifa base, el pago a los conductores y la parte que retiene la plataforma. Primero se comprueba hasta dónde llega un solo computador (Excel y Python); luego se resuelve con Hadoop MapReduce y con Spark, y se comparan.

Antes de empezar: Tu codespace debe pasar el verificador: bash /workspaces/tpd-codespace/tpd/verificar_codespace.sh termina en «RESULTADO: LISTO para el Taller 1».

Términos: si un término no te resulta claro, búscalo en Glosario_Tecnico_Tecnologias_Procesamiento_Datos.docx (carpeta Sesion_1_Vie_30_Octubre/Estudiantes).

El problema

La gerencia de operaciones necesita saber, con el mes completo de junio de 2025, cuántos viajes hizo cada empresa, cuánto cobró de tarifa base, cuánto pagó a sus conductores y qué parte se quedó la plataforma. Es la misma pregunta con la que Parrott y Reich (2018) sustentaron ante la TLC el pago mínimo a los conductores. El archivo tiene 19.868.009 viajes y pesa 4 GB: ¿con qué herramienta se responde completo, exacto y a tiempo?

Parte	Qué se resuelve
A	Comprobar hasta dónde llega un solo computador (Excel y pandas).
B	Resolverlo repartiendo el archivo y el cálculo con Hadoop MapReduce, y optimizar el shuffle.
C	Resolverlo con Spark, leyendo el CSV y el Parquet oficial, y comparar.
Final	La solución: la respuesta a la gerencia y qué herramienta usar.

Competencias que evidencia este taller

Competencia · resultado de aprendizaje	Dónde se evidencia	Evidencia que registras en la bitácora
C1 · RA1 — configurar un sistema Hadoop o Spark para procesar datos masivos con eficiencia y escalabilidad	Parte B, pasos 2 a 4 y 7; Parte C, pasos 2 y 6	Bloques y tareas del job, configuración de YARN que cambiaste y la curva de tiempo según los núcleos de Spark
C1 — diseñar sistemas distribuidos	Parte B, paso 8 (tu propio job)	Clave y valor que elegiste, por qué, y el resultado verificado
C4 · RA4 — ajustar configuraciones y parámetros para mejorar el rendimiento	Parte B, pasos 6 y 7; Parte C, paso 5	Bytes del shuffle con y sin combiner, tiempo con 4.096 y con 6.144 MB para YARN, CSV frente a Parquet

Cómo se trabaja en el codespace

Terminal	Cómo reconocerla	Para qué se usa
Terminal del codespace (Linux)	`@<tu usuario> ➜ …/tpd/hadoop (main) $`	Python, encender y apagar contenedores (scripts y docker compose) y ejecutar Spark (docker run).
Terminal del contenedor namenode (Linux)	`hadoop@namenode:~$`	Comandos de Hadoop: hdfs dfs y hadoop jar. Se entra en el paso 1 y se sale con exit.

- **Varias terminales:** para abrir otra terminal pulsa Ctrl + Mayús + ` (la tecla a la derecha de la Ñ) o el + del panel de abajo. Si el + parte la terminal en dos mitades, no pasa nada: cada mitad es una terminal. La lista de la derecha del panel muestra todas y permite pasar de una a otra con un clic. Cada bloque de comandos indica en cuál va («▶ Dónde»).
- **Copiar y pegar:** copia el comando del recuadro negro y pégalo con Ctrl+V. Después de cada comando, espera a ver de nuevo el indicador antes de pegar el siguiente.
- **Consolas web:** las páginas de Hadoop y de Spark no se abren con localhost. GitHub publica cada puerto en una dirección propia: `https://<nombre-de-tu-codespace>-PUERTO.app.github.dev`. Tu nombre de codespace es la primera parte de la dirección de la pestaña del editor (por ejemplo, scaling-carnival-4q7w667p4vgph5qwv). También puedes pulsar «Abrir en el navegador» en el aviso que aparece cuando un puerto se activa.

Consola	Puerto	Dirección
HDFS (NameNode)	9870	https://<nombre-de-tu-codespace>-9870.app.github.dev
YARN (ResourceManager)	8088	https://<nombre-de-tu-codespace>-8088.app.github.dev
Historial de jobs	19888	https://<nombre-de-tu-codespace>-19888.app.github.dev
Spark UI (mientras corre un job)	4040	https://<nombre-de-tu-codespace>-4040.app.github.dev

⚠ Si al abrir una consola aparece una página de GitHub pidiendo iniciar sesión o continuar, acéptala: la dirección es privada y solo la abre tu cuenta. Si se queda en «One second, just getting your codespace ready», recarga la página (F5).

Parte A — ¿Por qué no basta un computador?

En el caso: antes de usar herramientas distribuidas hay que demostrar que hacen falta. Se intenta responder la pregunta de la gerencia con lo que hay en cualquier oficina: Excel (lo muestra el docente) y Python con pandas en un solo computador.

Antes de empezar: encender Hadoop y lanzar la carga del mes a HDFS

Para qué: ganar tiempo: copiar el mes a HDFS tarda unos 4 minutos. Si lo lanzas ahora, la copia avanza mientras haces la Parte A.

En la terminal 1 enciende Hadoop con el script, que espera a que el clúster esté listo (incluido el modo seguro de HDFS), y lanza la carga:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $`

▶ **Qué hacer:** son 4 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 4

```
cd /workspaces/tpd-codespace/tpd/hadoop
```

Comando 2 de 4

```
bash encender_hadoop.sh
```

Comando 3 de 4

```
docker compose exec -T namenode hdfs dfs -mkdir -p /caso/input
```

Comando 4 de 4

```
docker compose exec -T namenode hdfs dfs -put /opt/datos/fhvhv_2025-06.csv \
    /caso/input/
```

Salida esperada *(los segundos cambian un poco en cada corrida)*

```
Encendiendo Hadoop (namenode, datanode, resourcemanager, nodemanager, historyserver)...
…
  NameNode responde                                    OK (31 s)
  DataNode registrado (Live datanodes 1)               OK (33 s)
  HDFS fuera del modo seguro (Safe mode is OFF)        OK (36 s)
  YARN con su nodo activo                              OK (37 s)

HADOOP LISTO en 37 s · consolas: HDFS puerto 9870 · YARN puerto 8088 (pestaña Puertos)
```

El put no imprime nada mientras copia (en la prueba tardó 4 minutos y 20 segundos). La copia terminó cuando vuelve el indicador. Para la Parte A abre una terminal 2 con Ctrl + Mayús + `.

⚠ No escribas nada en la terminal 1 mientras copia: lo que escribas ahí se pierde, no se ejecuta después. Tampoco la cierres.

Paso A1. Abrir el mes en Excel (lo muestra el docente)

Para qué: comprobar qué pasa cuando se abre con la herramienta de escritorio más usada un archivo que la supera.

El codespace no tiene Excel. El docente hace este paso en su computador con los mismos archivos; tú anota los resultados:

Lo que se encontró *(prueba del 27 de septiembre con Microsoft 365 de 64 bits)*

```
Mes (4,04 GB): Excel no lo abre (no abre archivos de texto de más de 2 GB)
Semana (0,93 GB): aviso «Los datos no caben en la cuadrícula»
Semana, última fila con datos: 1048576
pickup_datetime de esa fila: 2025-06-03 21:04:56
Viajes cargados: 1.048.575 de 4.576.158 (22,9 %)
```

Qué hace el docente, en su computador con Excel:

- Intenta abrir el mes (fhvhv_2025-06.csv, 4,04 GB): Excel no lo abre.
- Abre la semana (fhvhv_2025-06_semana.csv, 0,93 GB, el archivo que dejó el comando de la Preparación Previa): Excel avisa que los datos no caben en la cuadrícula.



Figura 1. Al abrir la semana, Excel avisa que los datos no caben y que perderá los que no cargó.

- Va a la última fila (Ctrl + flecha abajo) y mira la fecha de recogida (pickup_datetime) de ese viaje: hasta ahí llegó Excel.

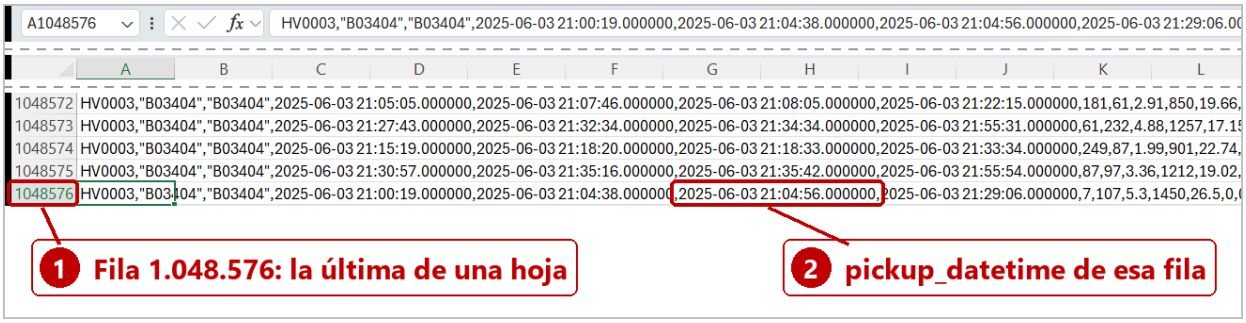


Figura 2. La fila 1.048.576 es la última de una hoja (1); el viaje de esa fila se recogió el 3 de junio a las 21:04 (2): de la semana del 2 al 8 de junio, Excel no pasó del 3.

Cómo leer el resultado: Excel tiene dos límites. El primero es el tamaño: no abre archivos de texto de más de 2 GB, así que el mes ni siquiera entra. El segundo son las filas: una hoja admite como máximo 1.048.576 (la primera es el encabezado). De la semana, Excel se quedó con el 2 de junio y casi todo el 3: menos de 2 de los 7 días. Cualquier suma hecha ahí describe solo esa porción, y ninguna celda lo advierte.

Paso A2. Intentarlo con Python (pandas)

Para qué: medir cuánta memoria pediría el mes completo en pandas, sin arriesgarse a bloquear el computador cargándolo entero.

`ensayo_python.py` carga el primer millón de viajes, mide cuánta memoria ocupan y proyecta el mes. Al final compara con la memoria RAM del codespace.

El script, parte por parte

Ábrelo en el explorador del codespace (panel izquierdo): `tpd/datos/ensayo_python.py`. Ubica cada parte:

- `pd.read_csv(ARCHIVO, nrows=MUESTRA, parse_dates=FECHAS)`: lee solo las primeras 1.000.000 filas (nrows) y convierte las cuatro columnas de fecha en fechas de 8 bytes, en vez de dejarlas como texto.
- `muestra.memory_usage(deep=True).sum()`: suma la memoria que ocupa cada columna; con deep=True cuenta también el contenido de cada texto.
- `proyeccion = gb * VIAJES_MES / MUESTRA`: regla de tres: los 19.868.009 viajes del mes ocupan unas 19,9 veces lo que ocupa el primer millón.
- `ram_del_equipo_gb()`: pregunta la memoria RAM física. En Windows usa GlobalMemoryStatusEx; en Linux, como el codespace, os.sysconf.
- `if proyeccion > ram * 0.6`: regla práctica: si el mes pediría más del 60 % de la RAM, no cabe con holgura, porque el sistema y los programas abiertos ya usan el resto.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
cd /workspaces/tpd-codespace/tpd/datos
```

Comando 2 de 2

```
python ensayo_python.py
```

Salida esperada *(codespace de 4 núcleos y 16 GB; los segundos cambian un poco)*

```
Archivo: fhvhv_2025-06.csv, 4.04 GB en disco
Primer millón de viajes: 0.54 GB en memoria, leídos en 3.6 s
Proyección del mes (19,868,009 viajes): 10.8 GB en memoria, y cerca de 1 minuto solo en leerlo
Memoria RAM de este equipo: 16.8 GB (el sistema y los programas abiertos ya usan una parte)
Conclusión: el mes no cabe con holgura en la memoria de este equipo.
```

Cómo leer el resultado: un millón de viajes ocupa cerca de medio GB en memoria, más que en disco, porque pandas guarda cada texto como un objeto de Python. El mes pediría unos 11 GB: en el codespace de 16 GB no cabe con holgura, y en un portátil de 8 GB, menos. Y un año son doce meses. La salida es repartir el archivo y el cálculo entre varias máquinas: eso hace Hadoop.

Sugerencia: en un computador con Windows y Python 3.11 la cifra es algo mayor (0,61 GB por millón y 12,0 GB el mes): la versión de Python cambia cómo se guardan los textos. La conclusión es la misma.

Parte B — MapReduce con Apache Hadoop

En el caso: como el mes no cabe con holgura en un computador, se reparte: HDFS guarda el archivo partido en bloques, YARN asigna memoria y núcleos a cada tarea, y MapReduce hace el cálculo en dos fases. En el map, cada viaje se convierte en (empresa, 1 viaje, tarifa, pago); en el reduce, se suman por empresa. El resultado es la respuesta a la gerencia: viajes, tarifa base, pago a los conductores y parte de la plataforma de Uber y de Lyft.

Prerrequisito: Hadoop encendido y la carga terminada

Vuelve a la terminal 1 y revisa que el put terminó (volvió el indicador). Si no encendiste Hadoop al empezar, hazlo ahora:

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
bash encender_hadoop.sh
```

✓ **Punto de control:** la salida termina en «HADOOP LISTO».

Abre las consolas de HDFS y de YARN (tabla de consolas, al inicio) y déjalas en pestañas: `https://<nombre-de-tu-codespace>-9870.app.github.dev` y `https://<nombre-de-tu-codespace>-8088.app.github.dev`.

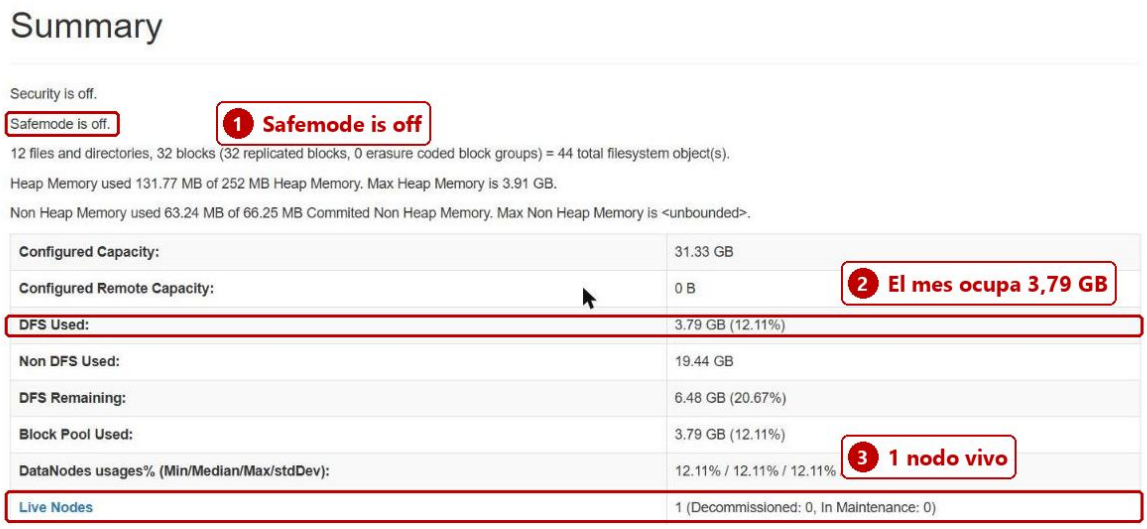


Figura 3. Consola de HDFS desde GitHub: «Safemode is off», 3,79 GB usados (el mes) y 1 nodo vivo (Live Nodes).

Paso 1. Entrar al contenedor namenode

Para qué: los comandos de Hadoop existen dentro de los contenedores, no en el codespace.

`docker compose exec namenode bash` abre una terminal Linux dentro del contenedor namenode. Dentro, `/opt/datos` es tu carpeta tpd/datos y `/opt/codigo` es tpd/hadoop (así lo define el docker-compose.yml).

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
docker compose exec namenode bash
```

✓ **Punto de control:** el indicador cambia a `hadoop@namenode:~$`.

Paso 2. Revisar el mes en HDFS

Para qué: comprobar que la carga quedó completa y en cuántos bloques la repartió HDFS: cada bloque será la entrada de una tarea map.

- `hdfs dfs -ls -h /caso/input` lista la carpeta con tamaños legibles (-h).
- `hdfs fsck … | grep …` revisa el archivo y deja solo las líneas del estado y del total de bloques.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
hdfs dfs -ls -h /caso/input
```

Comando 2 de 2

```
hdfs fsck /caso/input/fhvhv_2025-06.csv | grep -E "Total blocks|HEALTHY"
```

Salida esperada *(la fecha y la hora serán las de tu carga; HDFS usa la hora UTC)*

```
Found 1 items
-rw-r--r--   1 hadoop supergroup      3.8 G 2026-10-30 00:56 /caso/input/fhvhv_2025-06.csv
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&path=%2Fcaso%2Finput%2Ffhvhv_2025-06.csv
Status: HEALTHY
 Total blocks (validated):      31 (avg. block size 130394487 B)
The filesystem under path '/caso/input/fhvhv_2025-06.csv' is HEALTHY
```

Cómo leer el resultado: 4.042.229.115 bytes / 134.217.728 bytes por bloque = 30,1: HDFS usa 31 bloques (30 llenos y uno pequeño). El «1» después de los permisos es el factor de réplica: hay un solo DataNode, así que cada bloque tiene una sola copia (en un clúster real, el valor por defecto es 3). Anota los 31 bloques: serán 31 tareas map.

✓ **Punto de control:** el archivo aparece con 3.8 G y fsck dice HEALTHY con 31 bloques.

Paso 3. Probar el mapper y el reducer sin Hadoop

Para qué: encontrar errores de Python aquí, donde el mensaje se ve completo; dentro de Hadoop solo aparecería «subprocess failed».

La tubería (|) simula el job con las primeras 1.000 líneas: head las toma, el mapper las transforma, sort las ordena por clave (lo que Hadoop hace en el shuffle) y el reducer suma.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
head -n 1000 /opt/datos/fhvhv_2025-06.csv | python3 /opt/codigo/mapper.py | sort | \
    python3 /opt/codigo/reducer.py
```

Salida esperada

```
Lyft    229     5326.38 3835.01 28.0
Uber    770     19726.95        15852.39        19.6
```

Cómo leer el resultado: columnas: empresa, viajes, tarifa base (US$), pago a los conductores (US$) y parte de la tarifa que retiene la plataforma (%). 229 + 770 = 999 viajes: la línea 1.000 es el encabezado, que el mapper descarta.

Los scripts, parte por parte

`mapper.py` (ábrelo en el explorador: tpd/hadoop/mapper.py):

- `csv.reader(sys.stdin)`: separa cada línea en columnas respetando las comillas. Un split(",") a mano se desalinearía si algún campo trajera una coma.
- `if len(campos) != 25 or campos[0] == "hvfhs_license_num"`: descarta el encabezado y cualquier fila incompleta.
- `centavos(campos[11]), centavos(campos[18])`: toma base_passenger_fare y driver_pay y los pasa a centavos enteros, para que la suma sea exacta sin importar en qué orden la hagan las tareas.
- `reporter:counter:Caso viajes,Filas descartadas,1`: si un monto no es un número, suma 1 a un contador propio que aparece junto a los de Hadoop.
- `print(f"{empresa}\t1\t{tarifa}\t{pago}")`: emite la clave (Uber o Lyft), un TAB y los valores: 1 viaje, la tarifa y el pago.

`reducer.py`: recibe las líneas ordenadas por empresa y acumula mientras la empresa se repite; al cambiar, imprime la anterior. El bloque final imprime la última. Calcula la parte de la plataforma como (tarifa − pago) / tarifa. Con `--combinar` devuelve el formato del mapper, para usarse como combiner en el paso 6.

⚠ *El reducer asume que las claves llegan ordenadas; Hadoop lo garantiza tras el shuffle y en la prueba lo hace sort.*

Paso 4. Ejecutar el job con Hadoop Streaming

Para qué: procesar el mes completo repartido en 31 tareas map que YARN ejecuta en paralelo según la memoria disponible.

- `hadoop jar …hadoop-streaming-*.jar` ejecuta Hadoop Streaming, que permite usar scripts de Python como mapper y reducer.
- `-files /opt/codigo/mapper.py,/opt/codigo/reducer.py` copia los dos scripts a cada contenedor donde correrán las tareas.
- `-input … -output /caso/salida` archivo de entrada y carpeta de salida en HDFS. La carpeta de salida no debe existir: Hadoop la crea.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar -files \
    /opt/codigo/mapper.py,/opt/codigo/reducer.py -mapper "python3 mapper.py" -reducer \
    "python3 reducer.py" -input /caso/input/fhvhv_2025-06.csv -output /caso/salida
```

Mientras corre, la consola muestra el avance («map 0% reduce 0%» … «map 100% reduce 100%»); en el codespace tardó 3 minutos y 46 segundos. Al final imprime los contadores del job. Los que importan:

Salida esperada *(fragmento; el identificador del job cambia)*

```
INFO mapreduce.Job: Job job_…_0001 completed successfully
…
                Launched map tasks=31
                Map input records=19868010
                Map output records=19868009
                Combine input records=0
                Reduce shuffle bytes=369900781
                Reduce output records=2
…
real    3m46.586s
```

Ver el resultado: `hdfs dfs -cat` muestra el archivo de salida en la terminal.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter.

```
hdfs dfs -cat /caso/salida/part-*
```

Salida esperada

```
Lyft    5700648 145003586.84    113817449.39    21.5
Uber    14167361        411920420.29    316067761.82    23.3
```

✓ **Punto de control:** Uber tiene 14.167.361 viajes (71,3 %) y Lyft 5.700.648; la plataforma retiene cerca del 23 % (Uber) y del 21,5 % (Lyft) de la tarifa base.

Cómo leer los contadores

Contador	Valor	Qué significa
number of splits · Launched map tasks	31 · 31	Un split por bloque de HDFS y una tarea map por split: el paralelismo lo decide el tamaño del archivo.
Map input records	19.868.010	Todas las líneas del archivo: los viajes más el encabezado.
Map output records	19.868.009	Un par (empresa, valores) por viaje: el encabezado se descartó.
Combine input records	0	No hubo combiner: los 19,9 millones de pares viajaron tal cual al reducer.
Reduce shuffle bytes	369.900.781 (≈ 370 MB)	Lo que el reducer copió desde los mappers. Para producir dos líneas.
Reduce output records	2	Una línea por empresa.

Paso 5. Perfilar el job en las consolas

Para qué: ver dónde se fue el tiempo, con evidencia: leer, mover (shuffle) o sumar.

- En la consola de YARN (`https://<nombre-de-tu-codespace>-8088.app.github.dev`) abre la aplicación (columna ID) y pulsa «History»: se abre la página del job en el historial (puerto 19888).
- Anota Elapsed, Average Map Time, Average Shuffle Time y Average Reduce Time.

Lo que se midió *(corrida en el codespace)*

```
Elapsed: 3mins, 33sec
Average Map Time 6sec
Average Shuffle Time 1mins, 44sec
Average Merge Time 4sec
Average Reduce Time 21sec
```

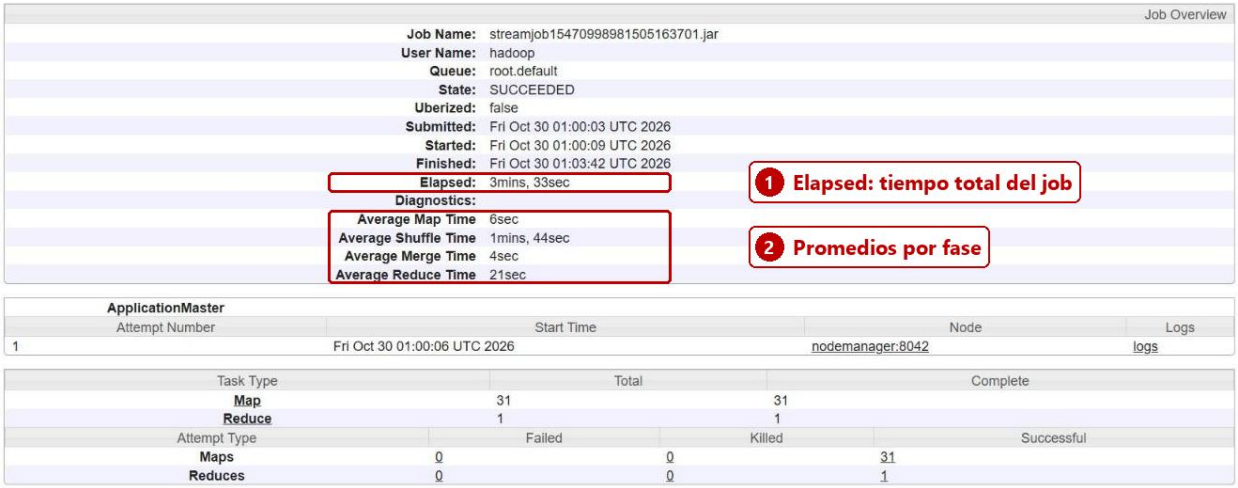


Figura 4. Página del job en el historial (puerto 19888): Elapsed y los promedios por fase.

Cómo leer el resultado: YARN tiene 4.096 MB: el ApplicationMaster ocupa 2.048 y cada tarea 1.024, así que corren dos maps a la vez y los 31 van por tandas. El «shuffle» del reducer incluye esperar a que termine el último map, por eso es el promedio más largo.

Paso 6. Optimizar el shuffle con un combiner

Para qué: reducir lo que viaja entre tareas: el combiner suma dentro de cada map antes del shuffle, así cada map envía dos líneas en lugar de cientos de miles.

`-combiner "python3 reducer.py --combinar"` usa el mismo reducer en modo combinador. Funciona porque sumar es asociativo.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar -files \
    /opt/codigo/mapper.py,/opt/codigo/reducer.py -mapper "python3 mapper.py" -combiner \
    "python3 reducer.py --combinar" -reducer "python3 reducer.py" -input \
    /caso/input/fhvhv_2025-06.csv -output /caso/salida_combiner
```

Comando 2 de 2

```
hdfs dfs -cat /caso/salida_combiner/part-*
```

Salida esperada *(fragmento de los contadores y resultado)*

```
INFO mapreduce.Job: Job job_…_0002 completed successfully
…
                Launched map tasks=31
                Combine input records=19868009
                Combine output records=62
                Reduce shuffle bytes=2344
…
hadoop@namenode:~$ hdfs dfs -cat /caso/salida_combiner/part-*
Lyft    5700648 145003586.84    113817449.39    21.5
Uber    14167361        411920420.29    316067761.82    23.3
```

	Sin combiner (paso 4)	Con combiner (paso 6)
Combine output records	—	62
Reduce shuffle bytes	369.900.781	2.344
Tiempo en el codespace	3 min 33 s	3 min 14 s
Resultado	Uber 14.167.361 · Lyft 5.700.648	idéntico

Cómo leer el resultado: 62 = 31 maps × 2 empresas: cada map envió dos líneas ya sumadas. El shuffle bajó de 370 MB a 2,3 KB con el mismo resultado al centavo. En un solo computador el tiempo total cambia poco, porque lo que más pesa es leer los 31 bloques; en un clúster real, donde el shuffle cruza la red entre máquinas, el ahorro pesa mucho más.

Paso 7. Ajustar la configuración de YARN y medir

Para qué: comprobar que la configuración del clúster, y no solo el código, decide el rendimiento (RA4), y encontrar el cuello de botella.

Antes de cambiar nada, responde en la bitácora: **si YARN tiene más memoria para repartir, caben más tareas a la vez; ¿esperas que el job tarde menos?, ¿por qué?**

1. Sal del contenedor con `exit` (vuelve el indicador del codespace).
2. En el explorador del codespace abre `tpd/hadoop/config` y cambia la línea `YARN-SITE.XML_yarn.nodemanager.resource.memory-mb=4096` por `…=6144`. Guarda con Ctrl+S.
3. Recrea solo los dos servicios de YARN (HDFS y lo cargado no se tocan) y vuelve a entrar al namenode:

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter.

```
exit
```

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
docker compose up -d --force-recreate resourcemanager nodemanager
```

Comando 2 de 2

```
docker compose exec namenode bash
```

Espera un minuto y recarga la consola de YARN: en Cluster Metrics, Total Resources debe decir <memory:6 GB, vCores:4>. Luego vuelve a ejecutar el job del paso 4 con otra carpeta de salida:

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar -files \
    /opt/codigo/mapper.py,/opt/codigo/reducer.py -mapper "python3 mapper.py" -reducer \
    "python3 reducer.py" -input /caso/input/fhvhv_2025-06.csv -output /caso/salida_yarn
```

Lo que se midió *(mismo job en el mismo codespace)*

```
Con 4.096 MB (paso 4): Elapsed 3 min 33 s
Con 6.144 MB (este paso): Elapsed 2 min 38 s, resultado idéntico
(la terminal mide un poco más porque incluye preparar el job: real 3m46s y 2m51s)
```

Cómo leer el resultado: el ApplicationMaster ocupa 2.048 MB y cada map 1.024. Con 4.096 MB solo caben 2 maps a la vez, y dos de los 4 núcleos del codespace quedan ociosos; con 6.144 MB caben 4 maps, uno por núcleo, y el mismo job bajó de 3 min 33 s a 2 min 38 s (cerca de un 26 % menos) con el resultado idéntico. El cuello de botella con 4.096 MB era la memoria que YARN podía repartir. Si se subiera más, ya no habría núcleos libres para más maps: ahí el límite pasarían a ser los núcleos. La lección de optimización: primero se identifica qué recurso falta y solo entonces se ajusta. Compara con lo que respondiste antes del cambio; tus segundos pueden variar entre corridas.

Al terminar, vuelve el valor a 4096 en `tpd/hadoop/config` (Ctrl+S) y recrea otra vez los servicios de YARN, para no afectar los siguientes pasos:

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter.

```
exit
```

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez.

Comando 1 de 2

```
docker compose up -d --force-recreate resourcemanager nodemanager
```

Comando 2 de 2

```
docker compose exec namenode bash
```

Paso 8. Diseña tu propio job: la espera por zona

Sigues dentro del contenedor namenode (indicador `hadoop@namenode:~$`), con YARN de vuelta en 4.096 MB.

Para qué: pasar de ejecutar un MapReduce a diseñarlo: decidir la clave, el valor y dónde se agrega. La gerencia pregunta: ¿en qué zonas esperan más los pasajeros desde que piden el viaje hasta que los recogen?

Antes de abrir los scripts, responde en tu bitácora: **¿cuál es la clave?, ¿qué valor emite el mapper?, y si se usa un combiner, ¿por qué no puede emitir el promedio de su bloque?**

Luego abre `tpd/hadoop/mapper_espera.py` y `reducer_espera.py` y compara con tu diseño:

- Clave: la zona de recogida (PULocationID). Valor: los segundos de espera (pickup_datetime − request_datetime) y un 1.
- Se emiten suma y conteo, no el promedio: el promedio de los promedios de cada bloque no es el promedio del mes. Sumas y conteos sí se pueden juntar por partes, así que el mismo reducer sirve como combiner con `--combinar`.
- El reducer divide al final y pone el nombre de la zona con zonas.csv (la tabla oficial de zonas de la TLC), que viaja con -files.

- Si un viaje tiene la recogida antes de la solicitud, el mapper no lo usa y suma 1 al contador «Esperas negativas».

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-*.jar -files \
    /opt/codigo/mapper_espera.py,/opt/codigo/reducer_espera.py,/opt/codigo/zonas.csv \
    -mapper "python3 mapper_espera.py" -combiner "python3 reducer_espera.py --combinar" \
    -reducer "python3 reducer_espera.py" -input /caso/input/fhvhv_2025-06.csv -output \
    /caso/espera
```

Comando 2 de 2

```
hdfs dfs -cat /caso/espera/part-* | awk -F'\t' '$3 >= 20000' | sort -t$'\t' -k4 -n \
    -r | head -n 5
```

Salida esperada *(columnas: zona, nombre, viajes, espera promedio en minutos)*

```
93      Flushing Meadows-Corona Park (Queens) 23764   10.84
117     Hammels/Arverne (Queens)      32624   8.27
86      Far Rockaway (Queens)  43788   7.95
118     Heartland Village/Todt Hill (Staten Island)   21129   6.76
217     South Williamsburg (Brooklyn) 35009   6.63
```

Cómo leer el resultado: 19.868.009 viajes − 231.332 con espera negativa = 19.636.677 pares emitidos: el contador propio «Esperas negativas» (página del job en el puerto 19888 → Counters) explica exactamente la diferencia. Esos 231.332 (1,2 %) son un hallazgo de calidad de datos que hay que reportar. El resultado coincide con un cálculo independiente sobre el Parquet: 261 zonas. En la prueba el job tardó 3 min 36 s. Entre las zonas con al menos 20.000 viajes, la mayor espera promedio del mes está en Flushing Meadows-Corona Park (10,8 min).

✓ **Punto de control:** tu tabla de las 5 zonas coincide con la de arriba y en tu bitácora quedó el diseño antes de ver los scripts.

Paso 9. Salir del contenedor y apagar el clúster

Para qué: liberar memoria para Spark, sin perder lo cargado en HDFS.

▶ **Dónde:** Terminal del contenedor namenode (bash), en la carpeta /opt/hadoop · el indicador se ve así: `hadoop@namenode:~$`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `hadoop@namenode:~$` (Ctrl+V o clic derecho) y presiona Enter.

```
exit
```

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> → /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> → /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter.

```
docker compose stop
```

⚠ *Usa stop, no down: docker compose down elimina los contenedores y con ellos todo lo cargado en HDFS.*

Si algo falla en la Parte B

Lo que ves	Causa	Qué hacer
`bash: hdfs: command not found`	El comando de Hadoop se pegó en la terminal del codespace y no dentro del contenedor.	Haz el paso 1 y repite.

Lo que ves	Causa	Qué hacer
`Cannot delete /tmp/hadoop-yarn/staging/…` o `0 datanode(s) running`	El job o el put se lanzaron antes de que HDFS estuviera listo.	Sal con exit, ejecuta bash encender_hadoop.sh, espera «HADOOP LISTO» y repite.
`put: `/caso/input/fhvhv_2025-06.csv': File exists`	El archivo ya se había subido.	No es un problema: sigue.
`Output directory hdfs://namenode/caso/salida already exists`	Se repitió el job: Hadoop no sobrescribe una carpeta de salida.	hdfs dfs -rm -r /caso/salida y vuelve a ejecutar.
`PipeMapRed.waitOutputThreads(): subprocess failed with code 1`	Error de Python en mapper.py o reducer.py.	Repite el paso 3: muestra el error completo.
El job se queda en «map 0% reduce 0%» varios minutos	YARN no tiene su nodo activo.	En la consola de YARN confirma Active Nodes = 1. Si es 0: exit, bash encender_hadoop.sh y repite.
«Killed map tasks» o «Killed reduce tasks» mayor que 0	Ejecución especulativa: YARN lanzó una copia de una tarea lenta.	No es un error: el resultado es correcto.

Parte C — El mismo cálculo con Apache Spark

En el caso: la misma pregunta de la Parte B, ahora con Spark, que procesa en memoria y lee directamente el Parquet oficial de la TLC. Se compara con Hadoop (resultado, tiempo y datos movidos) para decidir qué herramienta usar en un proceso por lotes sobre el mes completo.

Paso 1. Revisar el job, parte por parte

Abre `tpd/spark/job_spark.py` en el explorador:

- `esquema = StructType()…`: declara las 25 columnas con su tipo. Con inferSchema, Spark leería los 4 GB una vez más solo para adivinar los tipos.
- `F.when(F.col("hvfhs_license_num") == "HV0003", "Uber")…`: traduce el código de licencia al nombre de la empresa.
- `groupBy("empresa").agg(F.count("*"), F.sum(…cast("decimal(12,2)")))`: agrupa y suma: el equivalente del mapper + reducer, exacto al centavo.

Ese bloque (withColumn, groupBy, agg, orderBy) es la misma consulta que escribirías en SQL sobre una tabla de viajes:

```
SELECT CASE hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' END AS empresa,
       COUNT(*) AS viajes,
       SUM(base_passenger_fare) AS tarifa_base,
       SUM(driver_pay) AS pago_conductor,
       ROUND(100 * (SUM(base_passenger_fare) - SUM(driver_pay)) / SUM(base_passenger_fare), 1) AS
parte_plataforma_pct
FROM viajes
GROUP BY empresa
ORDER BY viajes DESC
```

Spark recibe esa misma lógica, la reparte en tareas sobre los bloques del archivo y suma por partes: lo que en Hadoop tuviste que escribir como mapper y reducer, aquí es una consulta de DataFrames (o de SQL) de pocas líneas.

- `resultado.show()` y `write.mode("overwrite").csv("output_spark")`: muestran el resultado y lo escriben en tpd/spark/output_spark. Son dos acciones: cada una recorre el archivo.
- `open("/dev/tty")`: deja la sesión abierta hasta que presiones Enter, para revisar la Spark UI. Funciona solo con docker run -it.

Paso 2. Ejecutar

Para qué: resolver la misma pregunta con el motor de Spark y medir cuánto tarda.

`-it` permite presionar Enter dentro del contenedor y `-p 4040:4040` publica la Spark UI (el codespace la reenvía a `https://<nombre-de-tu-codespace>-4040.app.github.dev`).

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd/hadoop · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $`

▶ **Qué hacer:** son 2 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd/hadoop (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 2

```
cd ..
```

Comando 2 de 2

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_spark.py
```

Salida esperada *(el tiempo cambia entre corridas)*

```
+-------+--------+------------+--------------+--------------------+
|empresa|viajes  |tarifa_base |pago_conductor|parte_plataforma_pct|
+-------+--------+------------+--------------+--------------------+
|Uber   |14167361|411920420.29|316067761.82  |23.3                |
|Lyft   |5700648 |145003586.84|113817449.39  |21.5                |
+-------+--------+------------+--------------+--------------------+

Tiempo total (incluye arranque de Spark): 78.8 s · núcleos: 4
Revisa la Spark UI en http://localhost:4040 y presiona Enter para cerrar Spark...
```

✓ **Punto de control:** las cifras son idénticas, al centavo, a las de Hadoop en el paso 4.

⚠ No presiones Enter todavía: la Spark UI solo existe mientras la sesión esté abierta. El mensaje dice http://localhost:4040, pero esa dirección no abre desde tu navegador: usa la del codespace (paso 3).

Paso 3. Analizar la ejecución en la Spark UI

Para qué: encontrar el shuffle de Spark y compararlo con el de Hadoop.

- Abre `https://<nombre-de-tu-codespace>-4040.app.github.dev` (o «Abrir en el navegador» en el aviso del puerto 4040) → pestaña «Jobs». Haz clic en la descripción azul del job 1 («showString», con «1/1 (1 skipped)» en Stages) y abre «DAG Visualization». El recuadro «Exchange» es el shuffle.
- Pestaña «Stages»: anota el Input del stage que lee el CSV, su número de tareas y el Shuffle Write.

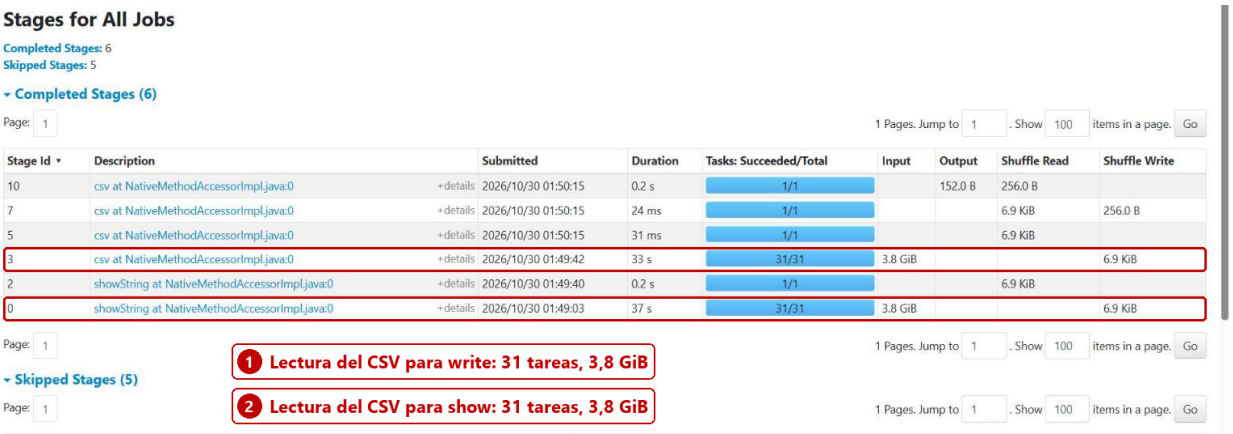


Figura 5. Spark UI → Stages: cada lectura del CSV usó 31 tareas, 3,8 GiB de Input y 6,9 KiB de Shuffle Write.

Lo que se midió *(valores de la corrida en el codespace)*

```
showString (primera acción): 31/31 tareas · Input 3.8 GiB · Shuffle Write 6.9 KiB · 37 s
csv (segunda acción, la escritura): 31/31 tareas · Input 3.8 GiB · Shuffle Write 6.9 KiB · 33 s
```

Cómo leer el resultado: Spark también partió el archivo en 31 tareas, pero su shuffle fue de unos 7 KB: suma por empresa dentro de cada tarea antes de enviar (agregación parcial), como el combiner del paso 6 de la Parte B. El CSV se leyó dos veces, una por cada acción (show y write), porque Spark no guarda resultados intermedios si no se le pide (cache).

- Cuando termines, vuelve a la terminal y presiona Enter para cerrar Spark. Espera a ver el indicador antes de pegar otro comando.

Paso 4. Revisar la salida

El resultado quedó en `tpd/spark/output_spark` (míralo en el explorador): un archivo part-….csv con encabezado y un archivo _SUCCESS vacío que indica que la escritura terminó bien.

Paso 5. El mismo job leyendo el Parquet oficial, sin convertir

Para qué: medir cuánto cambia el formato de almacenamiento. El CSV solo hizo falta para Excel y para Hadoop Streaming; Spark lee Parquet directamente.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $`

▶ **Qué hacer:** copia el recuadro negro completo, pégalo en la terminal a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $` (Ctrl+V o clic derecho) y presiona Enter. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_spark.py --parquet
```

Salida esperada *(fragmento)*

```
|Uber   |14167361|411920420.29|316067761.82  |23.3               |
|Lyft   |5700648 |145003586.84|113817449.39  |21.5               |

Tiempo total (incluye arranque de Spark): 35.9 s · núcleos: 4
```

- Antes de presionar Enter, abre la Spark UI → «Stages» y anota el Input y el número de tareas del stage que lee los datos.

Lo que se midió *(corridas en el codespace)*

```
CSV: 78,8 s · stage de lectura con 31 tareas · Input 3,8 GiB · Shuffle Write 6,9 KiB
Parquet: 35,9 s · stage de lectura con 4 tareas · Input 70,8 MiB · Shuffle Write 955 B
```

Cómo leer el resultado: el resultado es idéntico, pero Spark leyó unas 55 veces menos datos (70,8 MiB frente a 3,8 GiB) y tardó menos de la mitad. Parquet guarda por columnas, comprimido y con el esquema adentro: Spark leyó solo las 3 columnas que usa el cálculo y no tuvo que convertir texto a números. Elegir el formato es una decisión de optimización tan importante como el número de nodos.

Paso 6. Escalar: el mismo job con 1, 2 y 4 núcleos

Para qué: medir la escalabilidad (RA1): cuánto baja el tiempo al darle más núcleos a Spark. `--nucleos N` hace que Spark use N núcleos; el codespace tiene 4.

▶ **Dónde:** Terminal del codespace (bash), en la carpeta /workspaces/tpd-codespace/tpd · el indicador se ve así: `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $`

▶ **Qué hacer:** son 3 comandos, cada uno en su propio recuadro numerado. Copia y pega solo el recuadro 1 a la derecha de `@<tu usuario> ➜ /workspaces/tpd-codespace/tpd (main) $` (Ctrl+V o clic derecho) y presiona Enter; espera a que termine y

vuelva el indicador, y solo entonces pega el recuadro 2, y así sucesivamente. No copies dos recuadros a la vez. Un recuadro con varios renglones terminados en \ es un solo comando largo: cópialo completo, con todos sus renglones.

Comando 1 de 3

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_spark.py --parquet --nucleos 1
```

Comando 2 de 3

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_spark.py --parquet --nucleos 2
```

Comando 3 de 3

```
docker run --rm -it -p 4040:4040 -v "$PWD:/opt/tpd" -w /opt/tpd/spark \
    spark:3.5.9-python3 /opt/spark/bin/spark-submit job_spark.py --parquet --nucleos 4
```

Lo que se midió *(Parquet del mes completo; presiona Enter después de cada corrida)*

```
1 núcleo: 44,3 s
2 núcleos: 30,9 s
4 núcleos: 28,8 s
```

Cómo leer el resultado: de 1 a 2 núcleos el tiempo baja cerca de un tercio; de 2 a 4 casi no cambia. Hay una parte fija que no se reparte (arrancar Spark, planear el job, escribir la salida) y la lectura del Parquet tiene solo 4 tareas. Escalar sirve mientras haya trabajo que repartir; con un año de datos la curva seguiría bajando más. En la nube se escala agregando máquinas al clúster.

Si algo falla en la Parte C

Lo que ves	Causa	Qué hacer
`Path does not exist: file:/opt/tpd/datos/fhvhv_2025-06.csv`	El comando se ejecutó fuera de tpd.	cd /workspaces/tpd-codespace/tpd y repite.
`Bind for 0.0.0.0:4040 failed: port is already allocated`	Hay otra sesión de Spark abierta esperando Enter.	Vuelve a esa terminal y presiona Enter; luego repite.
La Spark UI no carga	Ya presionaste Enter y la sesión se cerró.	Repite el paso 2 y abre la UI antes de presionar Enter.
java.lang.OutOfMemoryError	El clúster de Hadoop sigue encendido y ocupa memoria.	Apágalo (paso 9 de la Parte B) y repite.

Paso 7. Comparar Hadoop y Spark con tus mediciones

Para qué: decidir con evidencia qué herramienta usar para este proceso: la misma pregunta, resuelta con dos motores y dos formatos.

Llena la tabla en tu bitácora con lo que mediste en la Parte B (pasos 4 y 6) y en la Parte C (pasos 2, 3 y 5). Los valores de abajo son los de la corrida en el codespace:

Criterio	Hadoop MapReduce	Spark leyendo el CSV	Spark leyendo el Parquet
Tiempo	3 min 33 s sin combiner; 3 min 14 s con combiner (historial)	78,8 s	35,9 s
Datos leídos	4,04 GB (31 bloques de HDFS)	3,8 GiB por cada acción (lo leyó dos veces)	70,8 MiB

Criterio	Hadoop MapReduce	Spark leyendo el CSV	Spark leyendo el Parquet
Datos movidos (shuffle)	370 MB sin combiner; 2.344 bytes con combiner	6,9 KiB	955 bytes
Tareas de lectura	31 maps	31	4
Resultado	Uber 14.167.361 · Lyft 5.700.648	idéntico al centavo	idéntico al centavo

- **Código:** en Hadoop escribiste dos programas (mapper.py, 18 líneas de código, y reducer.py, 24) y un comando largo; en Spark, el cálculo completo es un solo bloque de 12 líneas (withColumn, groupBy, agg) que equivale a una consulta SQL.
- **Resultado:** idéntico al centavo en los tres casos. La diferencia está en cuánto tarda, cuántos datos mueve y cuánto hay que programar.

Cómo leer el resultado: la pregunta es la misma; lo que cambia es el costo de responderla. MapReduce escribe a disco entre el map y el reduce, y sin combiner mueve cientos de MB por la red; Spark agrega por partes en memoria. Y el formato pesa tanto como el motor: con Parquet, Spark lee solo las columnas que necesita.

La solución

La respuesta a la gerencia, idéntica al centavo con Hadoop y con Spark (tarifa base y pago al conductor; sin peajes, impuestos ni propinas):

Empresa	Viajes	Tarifa base (US$)	Pago a conductores (US$)	Parte de la plataforma
Uber	14.167.361 (71,3 %)	411.920.420,29	316.067.761,82	23,3 %
Lyft	5.700.648 (28,7 %)	145.003.586,84	113.817.449,39	21,5 %

Herramienta	¿Resolvió el problema?	Evidencia del taller (codespace)
Excel	No: responde sobre una porción de los viajes sin advertirlo en las celdas.	No abre el mes; de la semana carga 1.048.575 de 4.576.158 viajes.
Python (pandas) en un computador	Solo si hay memoria de sobra, y deja de servir cuando los datos crecen.	Unos 11 GB para un mes en un computador de 16 GB: no cabe con holgura.
Hadoop MapReduce	Sí, repartiendo el archivo en bloques y tareas.	31 bloques, 31 maps; con combiner, el shuffle bajó de 370 MB a 2,3 KB.
Ajuste de YARN	Sí, cuando el recurso que falta es la memoria que YARN reparte.	Con 6.144 MB corrieron 4 maps a la vez (uno por núcleo) y el job bajó de 3 min 33 s a 2 min 38 s.
Spark	Sí, con menos escritura a disco, y escala con los núcleos.	El mismo resultado; con el Parquet oficial, unas 55 veces menos datos leídos.

Decisión técnica: para un proceso por lotes sobre el mes completo, Spark leyendo Parquet; MapReduce sigue siendo la base de HDFS y YARN sobre la que corre.

Paso 8. Cerrar la bitácora y apagar el codespace

En tu bitácora técnica registra, con la evidencia de las consolas: hasta dónde llegaron Excel y pandas; tiempos de Hadoop sin y con combiner y de Spark con CSV y con Parquet; bytes del shuffle en los tres casos; el efecto del ajuste de YARN y qué recurso era el cuello de botella; la curva de núcleos de Spark; y tu job de espera por zona.

⚠ Al terminar la sesión apaga el codespace: F1 → «Codespaces: Stop Current Codespace». Encendido sigue consumiendo horas de tu cuota.